
GORGO: Online Tuning for Cross-Region Network-Aware LLM Serving

Anonymous Author(s)

Affiliation

Address

email

Abstract

In cross-region LLM serving, time-to-first-token (TTFT) is highly sensitive to per-request routing decisions. KV-cache reuse across replicas can eliminate much of the prefill computation on multi-turn workloads, but realizing this potential requires jointly balancing cache locality against replica load and network cost. Additionally, public datasets to evaluate these policies do not represent long-context, high prefix-reuse workloads: LMSYS and WildChat average 2 and 2.5 turns per conversation, 0.5 and 2.9k tokens/request, and 5 and 9% intra-user reuse, respectively. To capture the setting where a majority of requests contain reusable prefixes, we evaluate common routing policies such as least-load, prefix-aware, session-affinity, and others on traces from a production inference endpoint featuring multi-turn conversations (~ 82 requests/user), $7\text{--}45\times$ longer prompts (21k avg tokens), and $2\text{--}6\times$ higher KV-cache reuse (55%). We evaluate policies on a multi-region GPU cluster where cross-region network latency ranges from 26ms to 466ms across replicas to test GORGO, a policy that jointly optimizes cache reuse, network latency, and queue depth with weights optimized online to minimize p95 TTFT with no offline profiling. GORGO achieves the lowest TTFT at every percentile (p50, p95, p99) across both a tuning window and a held-out evaluation window against all other load balancing policies, highlighting the impact of online joint optimization.

1 Introduction

Modern LLM serving is increasingly cross-region. Operators distribute model replicas across multiple geographies for three reasons: to follow diurnally shifting demand from a globally distributed user base, to provide redundancy against regional capacity loss or partition, and to aggregate accelerator capacity that no single region currently supplies [Xia et al., 2025, Mao et al., 2025]. Once replicas span regions, the gateway that routes incoming requests becomes a first-class component of TTFT, sitting between the client and the inference engines and choosing one replica per request based on observable state [AIBrix Team, 2024, F5 NGINX, 2024].

Time-to-first-token on a single replica decomposes into three terms: the network round-trip from the gateway to that replica, the queueing delay behind in-flight requests admitted via continuous batching [Yu et al., 2022], and the prefill computation for whatever portion of the input prompt is not already cached. Modern inference engines expose a per-replica radix-tree KV-cache that lets incoming requests skip prefill for any prefix already present on the handling replica [Zheng et al., 2024b, Kwon et al., 2023]. On long-context multi-turn traffic that saving can be the difference between sub-second TTFT and multi-second TTFT, so KV-cache locality at the routing layer is a first-order determinant of latency [Xia et al., 2025, Qin et al., 2024].

These three TTFT terms pull in different directions. Routing to the geographically nearest replica minimizes network RTT but ignores cache state; routing to the most cache-warm replica maximizes prefix reuse but can pile onto a saturated batch; routing to the least-loaded replica balances queue depth but is cache- and locality-blind. Standard production heuristics implement one of these signals at a time: NGINX-style least-request and least-connections [F5 NGINX, 2024], AIBrix-style longest-prefix-match [AIBrix Team, 2024], hash-based session affinity, and so on. They suffice for workloads with short prompts, weak prefix reuse, and uniform replica latencies, but interactive long-context traffic breaks all three assumptions: production chat traces concentrate most tokens in a small set of returning users with substantial prefix overlap, prompts run to tens of thousands of tokens, and cross-region RTTs differ by an order of magnitude. The right routing target for a given request can change minute-to-minute as load shifts, and weighting one signal in isolation leaves performance on the table at the tail.

A second obstacle is empirical: the public chat datasets used in prior LLM-serving evaluations underrepresent the regime where these trade-offs matter. LMSYS-Chat-1M [Zheng et al., 2024a] and WildChat-4.8M [Zhao et al., 2024] average 470 and 2,900 tokens per request with intra-user prefix reuse below 10%; the production endpoint we evaluate carries 21,000 tokens per request on average and 53.7% intra-user prefix reuse. Public datasets thus produce TTFT distributions that are nearly indistinguishable across cache-aware and cache-blind policies [Xia et al., 2025], which masks routing differences that production traffic exposes.

Contributions. (i) A workload characterization (§3) placing three datasets on the same prefix-reuse and request-length axes; the GLM-5.1 production trace has $7\text{--}45\times$ longer prompts and $2\text{--}6\times$ higher token-weighted reuse than LMSYS-Chat-1M and WildChat-4.8M. (ii) A routing cost model and three variants collectively called GORGO (§4) that score each replica by an additive combination of EWMA network RTT, estimated prefill cost over uncached tokens, and projected decode-rate cost over running and queued tokens. Variants differ only in how the two per-token rate parameters are set: a static configuration, a per-replica median-of-rates fit, or a Gaussian $(1+1)$ -ES hill climb that directly minimizes p95 TTFT. (iii) An experimental setup (§6) running each of eight policies on its own dedicated $2\times L40S$ three-replica fleet in Seoul, Frankfurt, and Ashburn, with cross-region RTTs spanning 26–466 ms. (iv) Empirical results (§7) across multiple 30-minute slices of GLM-5.1 traffic on two consecutive days plus a higher-concurrency stress window: in every window, two GORGO variants together cover the lowest TTFT at p50, p95, and p99 against seven baseline policies; on a WildChat replay the GORGO advantage collapses, confirming that the result is regime-dependent on long-context, high-reuse traffic.

2 Background

KV caching and prefix reuse. Inference engines such as SGLang [Zheng et al., 2024b] and vLLM [Kwon et al., 2023] maintain a Key–Value cache of attention activations for every active sequence; new tokens are appended to this cache step-by-step and admitted into the next decoding iteration via continuous batching [Yu et al., 2022]. Beyond per-request state, the same cache enables *prefix reuse*: when an incoming request shares a token-level prefix with a sequence already cached on a replica, that prefix’s KV state can be reused and prefill is skipped for those tokens. SGLang’s RadixAttention [Zheng et al., 2024b] operationalizes this via a per-replica radix trie that tracks which prefixes are resident; on multi-turn workloads where the same user returns repeatedly, a replica that has seen the conversation can serve the next turn at a fraction of the prefill cost.

Request-level routing as its own layer. Inside a single replica, a per-request scheduler decides admission and order within the running batch [Zheng et al., 2024b, Yu et al., 2022]; across replicas, a *request-level router* (or gateway) decides which replica receives a request in the first place. AIBrix [AIBrix Team, 2024] is a recent example of such a routing layer designed for LLM gateways, exposing prefix-aware, least-load, and least-request policies behind a common interface; production gateways like NGINX [F5 NGINX, 2024] and AWS Bedrock cross-region inference [Amazon Web Services, 2026] occupy the same architectural slot but at the network level, with no surface for KV state. Routing-layer decisions matter disproportionately on long-context workloads because the prefill term they implicitly select for can dominate TTFT [Xia et al., 2025, Qin et al., 2024].

Routing-cost decomposition. For request r routed to replica u , TTFT depends on three signal families the gateway can observe, each of which decomposes further: (1) *Network latency*: the

90 proxy-to-replica RTT $\text{rtt}(u)$ [Vulimiri et al., 2015]. (2) *Replica load*, comprising (2a) decode tokens
 91 of running requests in the current batch, (2b) prefill tokens of requests queued ahead of r , and (2c) r ’s
 92 own prefill cost once admitted. (3) *Available prefix cache*, comprising prefix overlap with (3a) the
 93 running batch (some KV state may already be evicted), (3b) queued requests (their KV state will
 94 exist by the time r runs), and (3c) r ’s own cached prior turns under the same conversation. A routing
 95 policy is a function over these nine sub-signals; no production policy we are aware of jointly weighs
 96 even all three top-level families.

97 **Common policies.** We consider five non-GORGON baselines representative of production systems:
 98 RANDOM (cache- and load-blind), LEAST-REQUEST and LEAST-LOAD (load-aware, cache-blind),
 99 PREFIX-CACHE (longest-prefix-match with a least-request fallback under load imbalance, in the
 100 AIBrix style [AIBrix Team, 2024]), and SIMPLE-SESSION-AFFINITY (token-prefix hash to a fixed
 101 replica). §5 states their exact form. None of the five takes more than one of the three signal families
 102 above as a primary objective.

103 **Production cross-region gateways and prior systems.** Industrial cross-region routing exists today
 104 but is opaque to KV state. AWS Bedrock cross-region inference [Amazon Web Services, 2026] and
 105 GKE multi-cluster Gateways [Google Cloud, 2026] provide global routing and failover but treat
 106 inference as an HTTP endpoint. Serverless GPU platforms such as Modal [Modal, 2026] support
 107 intra-region prefix caching but do not optimize across regions; SkyPilot [Yang et al., 2023] aggregates
 108 capacity across clouds but does not observe per-replica KV state. SkyServe [Mao et al., 2025]
 109 manages cross-region placement and SkyWalker [Xia et al., 2025] adds KV-aware cross-region load
 110 balancing with selective KV pushing. GORGON occupies the gap between opaque gateways and
 111 KV-aware single-region schedulers: a per-request, multi-region routing layer that combines all three
 112 signal families on a single scalar cost score, with the two scaling weights tuned online.

113 3 Dataset

114 **Public datasets sit in the wrong regime.** Routing policies that exploit cache locality or user
 115 stickiness improve TTFT by choosing a replica with high prefix-cache reuse, which we measure as
 116 the prefix-hit rate—the fraction of an incoming request’s input tokens that match a previously-seen
 117 prefix on the chosen replica. On a corpus where prefix-hit rates are uniformly low across all replicas,
 118 no cache-aware policy has anywhere to win. This is the regime occupied by the public chat datasets:
 119 LMSYS-Chat-1M [Zheng et al., 2024a] averages 467 input tokens per request and 9% global token-
 120 weighted prefix reuse, and WildChat-4.8M [Zhao et al., 2024] averages 2,925 input tokens and 5.3%
 121 intra-user reuse. The combination of short prompts and weak reuse pushes prefill cost down enough
 122 that TTFT distributions are nearly indistinguishable across cache-aware and cache-blind policies—an
 123 observation also made for cross-region serving more broadly [Xia et al., 2025]. Public benchmarks on
 124 these corpora consequently underestimate the gain from joint cache–network–queue-aware routing.

125 **GLM-5.1 production trace.** To measure routing in the regime where the trade-offs actually arise, we
 126 compile a long-context, multi-turn chatbot dataset from production inference traces of a commercial
 127 inference provider’s free GLM-5.1 endpoint [GLM Team, 2024]. The corpus contains 411,169
 128 requests over 7 days from 4,984 distinct users, yielding ~ 82 requests per user on average, with an
 129 average input length of 21,043 tokens and 53.7% intra-user token-weighted prefix reuse. Reuse is
 130 measured with a token-weighted prefix trie that ingests each request’s tokenized message sequence in
 131 arrival order, partitioned by user, and reports intra-user savings, cross-user (additional) savings, and
 132 global savings. Table 1 places the three datasets on the same axes; LMSYS does not carry an IP or
 133 user identity, so we treat each conversation as a separate user.

134 Three contrasts (Table 1, Fig. 1) matter for routing. *Request length:* GLM-5.1 prompts are $45\times$
 135 longer than LMSYS and $7\times$ longer than WildChat—at the prefill rates we measure, a 21,000-token
 136 uncached prompt costs seconds, not milliseconds. *User concentration:* GLM-5.1 has 82 requests
 137 per user; WildChat has 1.7; LMSYS has one. Intra-user reuse depends on returning users, which
 138 the public sets do not have at scale. *Reuse composition:* GLM-5.1’s reuse is intra-user multi-turn
 139 dialogue (53.7% intra plus 1.6% additional cross-user), while WildChat’s 34.4% global is mostly
 140 cross-user shared structure (5.3% intra). On GLM-5.1, a policy that lands a returning user on a replica
 141 that has seen their conversation can skip more than half of the prefill; on the public datasets, the same
 142 policy has nothing to skip.

Table 1: Dataset comparison along axes that determine the routing-decision surface. Token counts use the same byte-pair tokenizer in all rows. Reuse percentages are token-weighted prefix-trie savings. LMSYS-Chat-1M does not expose a stable user identifier (only conversation IDs), so its intra-user reuse and per-user-counts are reported as “—” (treating each conversation as a separate user yields essentially zero intra-user reuse).

Dataset	Total reqs	Users	Reqs/user	Avg in. tokens	Intra-user reuse	Global reuse
LMSYS-Chat-1M	1,000,000	—	—	467	—	9.0%
WildChat-4.8M	3,199,860	1,833,730	1.7	2,925	5.3%	34.4%
GLM-5.1 production	411,169	4,984	82.5	21,043	53.7%	55.3%

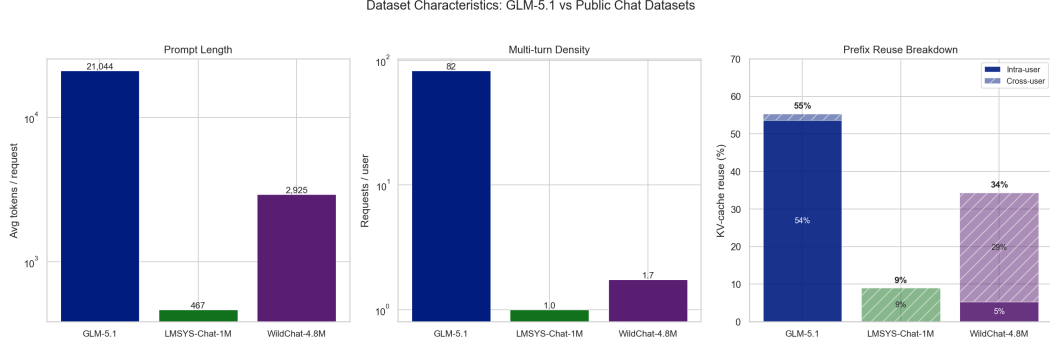


Figure 1: Three axes of dataset characterization. *Left*: average input tokens per request (log scale)—GLM-5.1 is $45\times$ longer than LMSYS and $7\times$ longer than WildChat. *Center*: requests per user (log scale)—82 on GLM-5.1 vs. 1.7 on WildChat vs. 1 on LMSYS. *Right*: prefix-reuse breakdown—GLM-5.1 reuse is dominated by intra-user repeat-conversation prefixes; WildChat reuse is dominated by cross-user structural similarity (shared system prompts, viral templates); LMSYS has no per-user signal at all.

Trace format and filtering. We re-encode GLM-5.1 into the Mooncake FAST’25 trace schema [Qin et al., 2024] so that benchmarking is time-sequenced and reproducible: each trace is a JSONL file in which every row carries an absolute-millisecond timestamp, request metadata (input/output token counts, target model), and the already-tokenized request body, which avoids exposing user data. Requests with $>24,000$ input tokens are filtered out; this is short of the model’s 32k native context window to leave headroom for our tokenizer disagreeing with the source tokenizer (so as to shrink the failed-request surface during testing). The filtered trace is then chunked by absolute timestamp into 30-minute windows; the windows we benchmark in §7 are drawn from April 1 and April 2 of this trace at the same time-of-day band. Replay is open-loop: requests are issued at the recorded arrival timestamps regardless of in-flight load, exposing each policy to production burstiness rather than to a closed-loop generator that would mask queuing pathologies.

4 GORGO

Cost model. Following the decomposition in §2, the GORGO score for routing request r to replica u is the sum of the three TTFT terms the gateway can observe directly:

$$\text{score}(u, r) = \text{rtt}_u + t_{\text{prefill}}(u) \cdot (|x_r| - |x_r \cap c_u|) + t_{\text{decode}}(u) \cdot (\text{queued_tokens}_u + \text{used_kv_tokens}_u), \quad (1)$$

where $|x_r| - |x_r \cap c_u|$ is the count of uncached input tokens for r on u , and $\text{queued_tokens}_u + \text{used_kv_tokens}_u$ is the combined token-weighted load already on u (queued tokens awaiting admission plus KV-cache footprint of running requests, both reported by SGLang). t_{prefill} and t_{decode} are per-replica per-token rate parameters with units of seconds-per-token: t_{prefill} converts uncached input-token count into expected prefill delay, t_{decode} converts ahead-of-line token load into expected decode/queueing delay. The proxy sends r to the replica with the minimum score. The two parameters are stored *per replica*, with global defaults overridden on any replica whose GPU class, batch size, or model variant differs; the online tuner writes per-replica overrides automatically because each TTFT

sample is labeled with its destination. Three signals feed the score: (i) rtt_u from an EWMA of a lightweight network probe decoupled from the metrics scrape, (ii) $|x_r \cap c_u|$ from the proxy’s local radix trie of cached prefixes per replica, refreshed on every request completion, and (iii) load from SGLang’s per-replica metrics combined with the proxy’s in-flight token counter.

Three weight-selection variants. The three GORGO variants share Equation 1 and differ only in how t_{prefill} and t_{decode} are chosen.

gorgo-static fixes both parameters at run start and never adjusts them. The starter values can be hand-set or the result of an offline characterization run on a previous window; the variant isolates the contribution of the cost model independent of online adaptation.

gorgo-autotune re-fits both rates online by linear regression through the origin on a trailing 64-sample window, partitioned by destination replica, every 16 new samples. For $t_{\text{prefill}}(u)$ we fit network-latency-normalized TTFT ($\text{TTFT} - \text{rtt}_u$) as a linear function of uncached input tokens; for $t_{\text{decode}}(u)$ we fit network-latency-normalized total completion time ($\text{TCT} - \text{rtt}_u$) as a linear function of completion tokens, decode-phase only. Implementation note: the median of per-sample rates we use is equivalent to a robust slope through the origin and tracks per-replica hardware drift. The variant treats the cost-model weights as identifiable physical rates rather than black-box knobs, which is also its failure mode—the per-sample rate $(\text{TTFT} - \text{rtt})/\text{uncached}$ inflates when uncached counts are small (high cache hit), so the median drifts upward.

gorgo-hillclimb treats the two parameters as black-box knobs and runs a Gaussian $(1+1)$ -evolution strategy [Rechenberg, 1973] over them in log-space, with the negative p95 TTFT of a rolling 64-request window as the objective. Every 16 new samples the tuner scores the current incumbent, accepts or rejects the most recent candidate, and proposes a new candidate $\theta_{\text{new}} = \exp(\log \theta_{\text{best}} + \sigma \mathcal{N}(0, I))$. The step size σ adapts according to Rechenberg’s 1/5-success rule [Rechenberg, 1973]: of the last 5 candidates evaluated, roughly one should improve the incumbent. We track the acceptance rate m over a trailing window and adjust σ multiplicatively—decreasing it (factor ~ 0.82) when $m > 1/5$ (the search is too tight to make progress) and increasing it (factor $\sim 1/0.82$) when $m < 1/5$ (the search is too loose and rarely finds improvements). The 1/5 rule is what makes online tuning practical at our timescales: matching empirical acceptance to 1/5 keeps the proposal scale calibrated for fast escape from poor regions early and refinement near the best-found incumbent late. On a 30-minute window with a 64-sample cohort and 16-sample hop, this drives the search to within a few percent of the best-found incumbent in roughly a hundred candidate evaluations (Fig. 3).

5 Baselines

We compare GORGO against five baseline policies.

random samples a replica uniformly per request; it is the lower bound.

least-request routes to the replica with the fewest in-flight requests, scoring by $\max(\text{sgLang_running_reqs}_u, \text{proxy_inflight}_u)$ to bridge the $\sim 10\text{s}$ staleness between metrics scrapes [F5 NGINX, 2024].

least-load routes to the replica with the lowest aggregate token-weighted load: $\text{running} + \text{queued} + \text{used_kv_tokens} + \text{queued_tokens}_u$. It is heavier-signal than least-request but cache-blind.

prefix-cache is the AIBrix-style longest-prefix-match policy [AIBrix Team, 2024]. It picks the replica with the longest cached prefix match and falls back to least-request when the running-request imbalance exceeds a soft threshold.

simple-session-affinity hashes the first 256 token IDs of the prompt modulo the number of replicas. The same prefix always maps to the same replica, maximizing intra-user reuse but providing no load-escape mechanism.

6 Experimental Setup

Hardware and software. Each policy runs on its own dedicated three-replica fleet so routing decisions for one policy cannot warm or cool another policy’s caches. Each fleet consists of one $2 \times \text{L40S}$ SGLang [Zheng et al., 2024b] server (tensor-parallel 2, 96 GB VRAM per replica) in each of

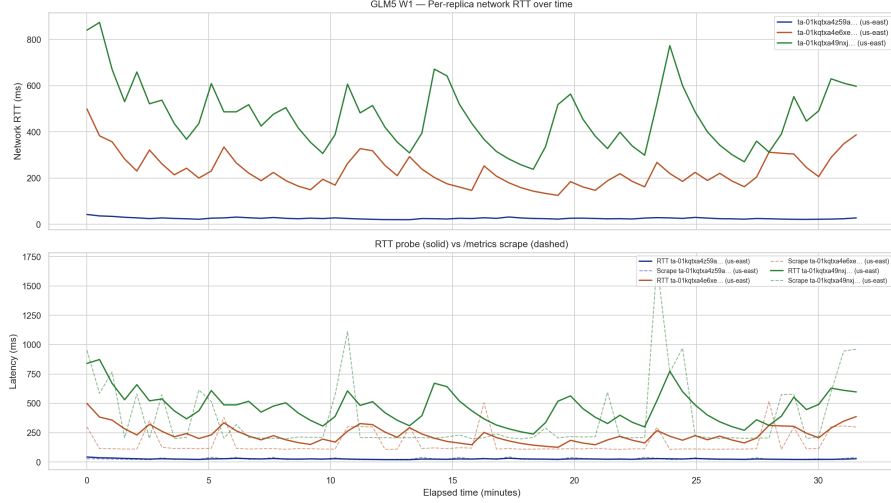


Figure 2: Cross-region RTT over W1 (probe from us-ashburn proxy). The local replica stays near 26 ms; the Frankfurt and Seoul legs vary by hundreds of ms within the window.

Asia (Seoul, ap-seoul), Europe (Frankfurt, eu-frankfurt), and US East (Ashburn, us-ashburn), serving Qwen3.5-35B-A3B-FP8 [Qwen Team, 2025]. A CPU proxy in US East hosts the routing policy and replays the trace. The eight policies (random, least-request, least-load, prefix-cache, simple-session-affinity, and the three GORGO variants) are benchmarked simultaneously on disjoint fleets so that any cross-region network variance during a window applies equally to all of them; GPUs are not shared across policies.

The proxy scrapes Prometheus metrics every ~ 10 s and runs a separate EWMA-smoothed HTTP probe for per-replica RTT, decoupled from the metrics scrape to avoid handler contention. Before each run a homogeneity check issues 16 streaming requests directly to each replica; runs whose worst-to-median TTFT ratio exceeds $3\times$ are flagged (none occurred).

Windows. We benchmark across multiple 30-minute slices of the GLM-5.1 Mooncake trace covering April 1 and April 2, plus a higher-concurrency stress slice on April 2:

- **W1 (April 1, 00:30–01:00 UTC, $c=32$):** tuning window. GORGO variants seed from manually-chosen starter weights and the online tuners adapt during the window.
- **W2 (April 2, 00:30–01:00 UTC, $c=32$):** cross-day evaluation window. Same time-of-day band as W1, different day. gorgo-static deploys the per-replica weights that gorgo-hillclimb learned in W1 and never adjusts them; the other GORGO variants continue tuning.
- **Stress (April 2, 00:30–01:00 UTC, $c=64$):** same trace as W2 replayed at concurrency 64 to test behavior under elevated load.

Concurrency c is the in-flight-request cap at the proxy. Output is capped at 128 tokens per request so TTFT, not decode, dominates the latency budget.

Cross-region RTT. Probed from the US East proxy with a dedicated lightweight HTTP probe over the duration of W1, mean EWMA-smoothed RTT is 25.6 ± 3.8 ms to the local US East replica, 232 ± 72 ms to Frankfurt, and 466 ± 138 ms to Seoul; the high coefficient of variation on the cross-region legs (29–31%) reflects routing-table churn during the window. Figure 2 plots the time series.

Metrics. For each policy and window we report TTFT p50, p95, and p99 over all successful streaming responses. Failed requests (HTTP 4xx/5xx, dominated by prompt-too-long at the effective context boundary) are excluded from percentiles but counted in success rate.

Table 2: GLM-5.1 W1 (tuning window, 00:30–01:00 UTC). TTFT in seconds. Bold is best in column. Completed counts from proxy logs; success rate ≈ 84.2 – 84.6% .

Policy	Completed	p50	p95	p99	max	Succ.%
gorgo-hillclimb	4,065	0.288	1.466	2.307	4.39	84.4
simple-session-affinity	4,054	0.329	1.482	2.804	14.96	84.2
gorgo-static	4,076	0.299	1.605	2.357	4.16	84.6
least-request	4,074	0.293	1.632	2.355	13.71	84.6
prefix-cache	4,075	0.297	1.654	3.017	5.96	84.6
gorgo-autotune	4,076	0.294	1.658	2.428	4.18	84.6
least-load	4,066	0.297	1.728	2.629	4.64	84.4
random	4,076	0.296	1.828	3.171	28.99	84.6

Table 3: GLM-5.1 W2 (April 2 00:30–01:00 UTC, $c=32$, cross-day evaluation). *gorgo-static* deploys the W1-learned per-replica weights and does not adjust them. Bold is best in column.

Policy	Completed	p50	p95	p99	max	Succ.%
gorgo-static	2,462	0.329	1.387	1.947	3.46	89.4
gorgo-hillclimb	2,468	0.283	1.416	2.118	4.14	89.6
simple-session-affinity	2,462	0.327	1.435	2.039	2.88	89.4
gorgo-autotune	2,466	0.264	1.576	2.306	3.24	89.5
random	2,461	0.282	1.610	2.306	3.18	89.4
least-request	2,465	0.285	1.622	2.293	3.31	89.5
least-load	2,467	0.279	1.690	2.364	3.35	89.6
prefix-cache	2,460	0.277	1.740	2.530	15.01	89.3

7 Results

In W1 (Table 2), *gorgo-hillclimb* delivers the lowest p50 (0.288 s), p95 (1.466 s), and p99 (2.307 s)—a single GORGO variant sweeps every percentile during tuning. *gorgo-static* (starter weights, never adjusted) already beats every non-GORGO policy on p99 and ranks third on p95, showing the cost model alone improves on single-signal heuristics; *gorgo-hillclimb* extends the lead by adapting per-replica rates online. *prefix-cache* is competitive at p95 but tail-degraded at p99 (3.017 s) by concentrating traffic onto warm-cache replicas; *simple-session-affinity*’s p99 (2.804 s) and max (14.96 s) reflect transient overload of the hashed replica. Starting from $t_{\text{prefill}} = 0.07$, $t_{\text{decode}} = 0.06$, the ES converged to $t_{\text{prefill}} \approx 0.98$, $t_{\text{decode}} \approx 4.4 \times 10^{-4}$; the high t_{prefill} reflects $\sim 9.5\text{k}$ uncached tokens per request after 55% reuse, and the low t_{decode} reflects that deep queues are rare at $c=32$.

In W2 (Table 3, same time-of-day as W1 on a different day), *gorgo-static*—deploying the W1-learned weights without adjustment—wins p95 (1.387 s) and p99 (1.947 s); *gorgo-autotune* wins p50 (0.264 s). Two GORGO variants together sweep all three percentiles. *gorgo-static*’s W2 p95 improves 13.6% on its W1 p95 and 5.4% on *gorgo-hillclimb*’s W1 p95, with frozen weights—an operator can run *gorgo-hillclimb* during a tuning window and freeze the result into *gorgo-static* for production. *prefix-cache*’s p95 *inflates* 5.2% from W1 to W2; a longest-prefix-match policy that never re-balances under load drifts in the wrong direction across days.

At $c=64$ (Table 4), the picture flips between *gorgo-static* and *gorgo-hillclimb*: *gorgo-static* wins p50 (0.179 s) but loses p95/p99 to *gorgo-hillclimb* (1.331 s and 1.832 s); the two together still sweep all three percentiles. *gorgo-hillclimb* improves 6.0% on p95 and 13.5% on p99 versus its $c=32$ W2 result as the ES adapts to higher load; the W1-frozen weights are no longer optimal at $c=64$.

Online-tuning convergence. Figure 3 plots the best-cache-pick rate—the fraction of requests routed to the replica that already holds the most of the request’s input prefix—over the W1 tuning window for five policies. *gorgo-hillclimb* converges to picking the best-cached replica on ~ 95 – 100% of requests within the first ~ 2 minutes of online tuning and stays there for the remainder of the window; the other policies either oscillate (*prefix-cache*) or remain at lower rates (*simple-session-affinity*, *random*, *gorgo-static*). This is the empirical signature of the (1+1)-ES with the 1/5 rule: fast initial descent during the first few hop cycles, then stable refinement.

Table 4: GLM-5.1 Stress (April 2 00:30–01:00 UTC, $c=64$). Same trace as W2, replayed at $2\times$ the in-flight-request cap. Bold is best in column. `simple-session-affinity` did not complete its sweep at $c=64$ and is omitted.

Policy	Completed	p50	p95	p99	max	Succ.%
gorgo-hillclimb	2,466	0.281	1.331	1.832	3.05	89.5
prefix-cache	2,468	0.288	1.394	1.992	2.95	89.6
gorgo-static	2,472	0.179	1.427	2.302	4.06	89.8
least-request	2,465	0.281	1.605	2.268	4.16	89.5
gorgo-autotune	2,469	0.280	1.690	2.298	18.84	89.7
least-load	2,465	0.281	1.700	2.290	2.98	89.5
random	2,460	0.287	1.724	2.357	4.00	89.3

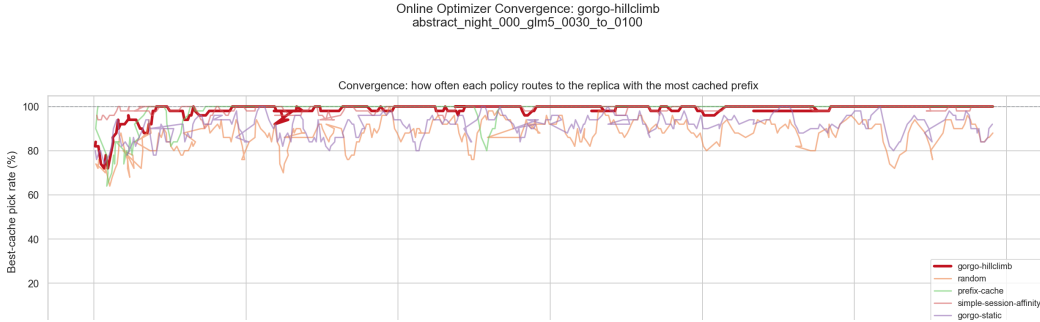


Figure 3: Best-cache-pick rate over the W1 tuning window for five policies. `gorgo-hillclimb` (red) converges to $\sim 95\text{--}100\%$ within the first few minutes; baselines either oscillate or stay flat at lower rates.

273 **Per-policy summary across W2.** Figure 4 adds a per-policy view of W2 (cross-day evaluation):
 274 top-left is p50/p95/p99 sorted by p95, top-right is routing concentration (share of requests on the
 275 most-used replica), bottom panels plot smoothed median and p95 TTFT over the 30-minute window.
 276 `gorgo-static` (frozen W1 weights) ranks first on p95 with `gorgo-hillclimb` second; smoothed-
 277 p95 traces show gorgo variants are also visibly more stable across the window than baselines.

278 **Regime sensitivity (WildChat replay).** A control replay on WildChat-4.8M with the same fleet
 279 and proxy reverses the ranking: `simple-session-affinity` wins p95 (1.025 s) and p99 (1.712 s),
 280 `least-request` is competitive (1.036 s p95), and the GORGO variants are between 30% and 250%
 281 worse on p95 (`gorgo-hillclimb` 1.325 s, `gorgo-static` 1.932 s, `gorgo-autotune` 3.583 s). On
 282 a corpus where prompts average 2,900 tokens and intra-user reuse is 5.3%, the prefill term that
 283 GORGO’s cost model is most sensitive to is small relative to network and queue terms, and the
 284 per-token rate parameters do not converge during a 30-minute window. The advantage GORGO
 285 shows on GLM-5.1 is therefore regime-dependent on long-context, high-reuse traffic; we discuss this
 286 in §8.

287 8 Limitations

288 **Statistical scope.** Three 30-minute windows from one production trace; each policy run once per
 289 window. With $\sim 2,500\text{--}4,800$ successful responses, p99 standard error is $\sigma_{\text{TTFT}}/\sqrt{n \cdot 0.01 \cdot 0.99} \approx$
 290 $0.05\text{--}0.07$ s. GORGO p95/p99 wins are several SE clear of the next baseline in W2 and Stress; in W1
 291 the p99 lead is ~ 1 SE. Cross-window replication on additional time-of-day slices remains future work.
 292 **Regime sensitivity.** The WildChat replay (§7) shows GORGO’s advantage is specific to long-context,
 293 high-reuse traffic: on prompts averaging 2,900 tokens with 5.3% intra-user reuse, the prefill term
 294 is small, the per-token rates do not converge in 30 minutes, and `simple-session-affinity` wins.
 295 We claim improvement in the regime characterized in §3, not universally. **Methodological and**
 296 **operational caveats.** `gorgo-autotune` is the weakest GORGO variant on GLM-5.1 because the per-
 297 sample rate $(\text{TTFT} - \text{rtt})/\text{uncached}$ inflates at high cache hit, biasing the fit. `gorgo-hillclimb`
 298 requires several hop cycles to converge; this fits comfortably in 30-minute windows but not in

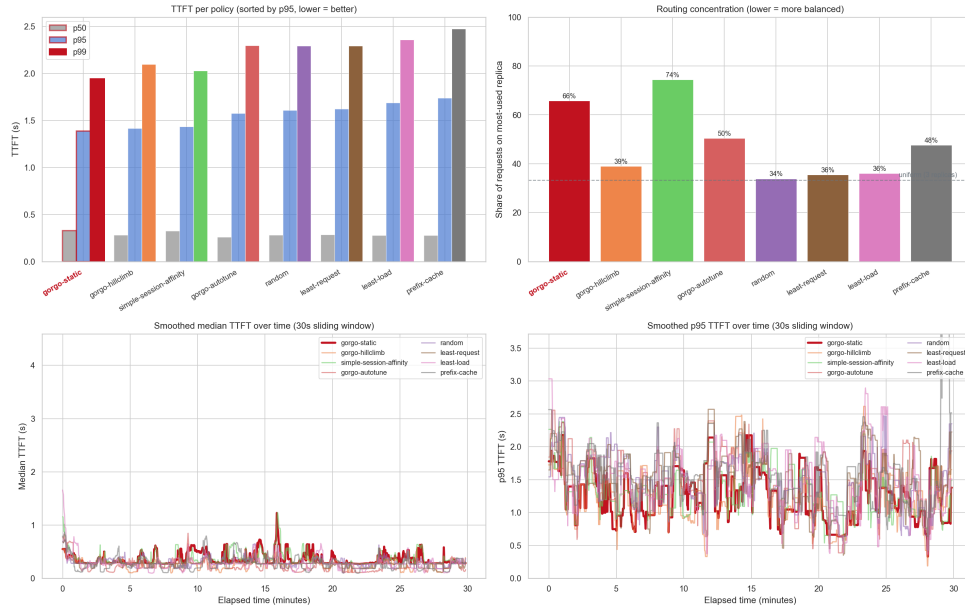


Figure 4: Per-policy summary over W2 (April 2 cross-day, $c=32$). Top: TTFT percentiles (sorted by p95) and routing concentration. Bottom: smoothed median and p95 TTFT over the 30-minute window.

arbitrarily short ones. Each policy ran on a single-tenant three-replica fleet; the centralized proxy is a single point of failure, and a distributed peer-summary GORGO variant is sketched but not benchmarked. The fleet uses homogeneous GPUs and does not split prefill/decode [Zhong et al., 2024, Patel et al., 2024], which would require an additional routing term; GORGO uses only HTTP-exposed metrics, not scheduler-internal state [Pan et al., 2025, Yang et al., 2025a] that could tighten the prefill estimate.

9 Related Work

Prefix caches and reuse. RadixAttention in SGLang [Zheng et al., 2024b] stores per-request KV state in a radix tree; PagedAttention in vLLM [Kwon et al., 2023] enables prefix sharing through paged memory. KVLink [Yang et al., 2025b], ChunkKV [Liu et al., 2025], KVFlow [Pan et al., 2025], and Learned Prefix Caching [Yang et al., 2025a] improve the single-replica cache but do not decide which replica serves a request. **Routing and scheduling.** Mooncake [Qin et al., 2024] is a KV-cache-centric architecture and the source of our trace format. AIBrix [AIBrix Team, 2024] provides prefix-aware load balancing as a baseline; k-LPM [Dexter et al., 2025] formulates LLM scheduling under TTFT constraints as NP-hard; DLPM [Cao et al., 2025] targets fairness with locality; Llumnix [Sun et al., 2024] migrates requests across replicas; Multi-LLM routers [Wu and Silwal, 2025] address model selection; CacheBlend [Yao et al., 2024] routes by trie-measured prefix length but does not measure network RTT. GORGO commits to a single scalar score per replica with online weight tuning and requires no engine modifications. **Online and learned routing.** Performance-aware LLM load balancing [Jain et al., 2025] learns a routing policy over mixed prefill/decode workloads; bandit and RL schedulers [Mao et al., 2019] and Bayesian-optimization configuration tuners [Alipourfard et al., 2017] explore the broader space. GORGO keeps the policy interpretable (an additive cost) and applies (1+1)-ES only to two scaling weights, with TTFT as direct reward. **PD-disaggregation.** Sarathi-Serve [Agrawal et al., 2024], DistServe [Zhong et al., 2024], and Splitwise [Patel et al., 2024] disaggregate prefill from decode at the batch or GPU-pool level. These reduce the compute component of TTFT but do not resolve geo-routing trade-offs when locality and network latency conflict; GORGO is complementary and assumes an underlying stack that may already use either.

10 Conclusion

We characterize a workload regime—long-context, multi-turn, high prefix-reuse production traffic—where standard LLM-serving routing heuristics leave performance on the table because they optimize one of network RTT, replica load, or cache locality and ignore the other two. On a 411,000-request production trace where 53.7% of input tokens are part of a previously-seen prefix from the same user, GORGO—a single-score additive cost model with two per-token rate parameters tuned online—collectively achieves the lowest TTFT at every percentile (p50, p95, p99) across a tuning window, a cross-day evaluation window, and a higher-concurrency stress window on a 2×L40S three-region fleet with cross-region RTTs spanning 26–466 ms. Per-percentile wins are not held by any single fixed weight setting: gorgo-hillclimb’s online (1+1)-ES dominates during tuning, the W1-learned weights frozen into gorgo-static dominate under cross-day evaluation, and the two together cover all percentiles under stress. A WildChat replay demonstrates that the result is regime-dependent on the long-context, high-reuse axis, not a universal win. The cost model and tuner fit on a stock HTTP gateway and require no offline profiling.

References

- A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, R. Ramjee, and A. Tumanov. Sarathi-Serve: Llm inference made efficient and fair. *arXiv preprint*, 2024.
- AI Brix Team. AIBrix: An open-source infrastructure for LLM inference at scale. <https://github.com/vllm-project/aibrix>, 2024.
- Omid Alipourfard, Hongqiang Harry Liu, Jianshu Chen, Shivaram Venkataraman, Minlan Yu, and Ming Zhang. CherryPick: Adaptively unearthing the best cloud configurations for big data analytics. In *Proceedings of the 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI ’17)*, 2017.
- Amazon Web Services. Increase throughput with cross-region inference. Documentation, 2026. Accessed 2026-01-28.
- Shiyi Cao, Yichuan Wang, Ziming Mao, Pin-Lun Hsu, Liangsheng Yin, Tian Xia, Dacheng Li, Shu Liu, Yineng Zhang, Yang Zhou, Ying Sheng, Joseph Gonzalez, and Ion Stoica. Locality-aware fair scheduling in LLM serving. *arXiv preprint arXiv:2501.14312*, 2025.
- Gregory Dexter, Shao Tang, Ata Fatahi, Qingquan Song, Tejas Dharamsi, and Aman Gupta. LLM query scheduling with prefix reuse and latency constraints. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- F5 NGINX. NGINX documentation: Load balancing methods. <https://docs.nginx.com/nginx/admin-guide/load-balancer/http-load-balancer/>, 2024.
- GLM Team. GLM-4: A family of open multilingual multimodal chat LLMs, 2024.
- Google Cloud. About multi-cluster gateways. Documentation, 2026. Accessed 2026-01-28.
- Kunal Jain, Anjaly Parayil, Ankur Mallick, Esha Choukse, Xiaoting Qin, Jue Zhang, Íñigo Goiri, Rujia Wang, Chetan Bansal, Victor Rühle, Anoop Kulkarni, Steve Kofsky, and Saravan Rajmohan. Performance aware LLM load balancer for mixed workloads. In *Proceedings of the 5th Workshop on Machine Learning and Systems (EuroMLSys ’25)*, 2025.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2023.
- Xiang Liu, Zhenheng Tang, Peijie Dong, Zeyu Li, Yue Liu, Bo Li, Xuming Hu, and Xiaowen Chu. ChunkKV: Semantic-preserving KV cache compression for efficient long-context LLM inference. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- H. Mao, Z. Yang, A. Phanishayee, R. Mahajan, and I. Stoica. SkyServe: Serving ai models across regions and clouds with spot instances. In *Proceedings of the 2025 USENIX Annual Technical Conference*, 2025.

374 Hongzi Mao, Malte Schwarzkopf, Shaileshh Bojja Venkatakrisnan, Zili Meng, and Mohammad
375 Alizadeh. Learning scheduling algorithms for data processing clusters. In *Proceedings of the ACM*
376 *Special Interest Group on Data Communication (SIGCOMM '19)*, 2019.

377 Modal. Prefix caching. Documentation, 2026. Accessed 2026-01-28.

378 Zaifeng Pan, Ajikumar Patel, Zhengding Hu, Yipeng Shen, Yue Guan, Wan-Lu Li, Lianhui Qin,
379 Yida Wang, and Yufei Ding. KVFlow: Efficient prefix caching for accelerating LLM-based
380 multi-agent workflows. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
381 arXiv:2507.07400.

382 Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo
383 Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *ACM/IEEE*
384 *International Symposium on Computer Architecture (ISCA)*, 2024.

385 Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yanzhe Wu, Weimin Zheng, Xinran Chen,
386 Rui Xu, Xingda Yao, Rongxin Wei, Zhiyuan Zhao, Jing Huang, Mingjie Yang, and Jidong Wu.
387 Mooncake: A KVCache-centric disaggregated architecture for LLM serving, 2024.

388 Qwen Team. Qwen3 technical report. 2025. Alibaba Cloud.

389 Ingo Rechenberg. Evolutionsstrategie: Optimierung technischer systeme nach prinzipien der biolo-
390 gischen evolution. 1973.

391 Biao Sun, Ziming Huang, Hanyu Tang, Chengquan Wang, Cheng Zhang, Yiming Li, Liu Yang,
392 Wencong Zhang, Lin-Wei Wang, Tianhe Wu, Ang Cao, Yongqiang Lin, et al. Llumnix: Dynamic
393 scheduling for large language model serving. In *USENIX Symposium on Operating Systems Design*
394 *and Implementation (OSDI)*, 2024.

395 Ashish Vulimiri, Carlo Curino, P. Brighten Godfrey, Thomas Jungblut, Jitendra Padhye, and George
396 Varghese. Global analytics in the face of bandwidth and regulatory constraints. In *Proceedings of*
397 *the 12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*, 2015.

398 Fangzhou Wu and Sandeep Silwal. Efficient training-free online routing for high-volume multi-LLM
399 serving. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

400 Tian Xia, Ziming Mao, Jamison Kerney, Ethan J. Jackson, Zhifei Li, Jiarong Xing, Scott Shenker,
401 and Ion Stoica. SkyWalker: A locality-aware cross-region load balancer for LLM inference. arXiv
402 preprint arXiv:2505.24095, 2025.

403 Dongsheng Yang, Austin Li, Kai Li, and Wyatt Lloyd. Learned prefix caching for efficient LLM
404 inference. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025a.

405 Jingbo Yang, Bairu Hou, Wei Wei, Yujia Bao, and Shiyu Chang. KVLink: Accelerating large
406 language models via efficient KV cache reuse. *arXiv preprint arXiv:2502.16002*, 2025b.

407 Zongheng Yang et al. SkyPilot: An intercloud broker for sky computing. Preprint, 2023.

408 Jiayi Yao, Hanchen Li, Yuhao Liu, Siddhant Ray, Yihua Peng, Hao Zhang, Ion Stoica, Kurt Keutzer,
409 and Michael W. Mahoney. CacheBlend: Fast large language model serving for RAG with cached
410 knowledge fusion, 2024.

411 Gyeong-In Yu, Jinseok Jeong, Gyeongchan Kim, Suhyun Kim, and Byung-Gon Chun. Orca: A
412 distributed serving system for transformer-based generative models. In *Proceedings of the 16th*
413 *USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022.

414 Wenting Zhao, Xiang Ren, Jack Hessel, Claire Cardie, Yejin Choi, and Yuntian Deng. WildChat: 1M
415 ChatGPT interaction logs in the wild. In *International Conference on Learning Representations*
416 *(ICLR)*, 2024.

417 Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Tianle Li, Siyuan Zhuang, Zhanghao Wu, Yonghao
418 Zhuang, Zhuohan Li, Zi Lin, Eric P. Xing, Joseph E. Gonzalez, Ion Stoica, and Hao Zhang. LMSYS-
419 Chat-1M: A large-scale real-world LLM conversation dataset. In *International Conference on*
420 *Learning Representations (ICLR)*, 2024a.

- 421 Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi
422 Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang:
423 Efficient execution of structured language model programs. In *Advances in Neural Information*
424 *Processing Systems (NeurIPS)*, 2024b.
- 425 Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao
426 Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language
427 model serving. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*,
428 2024.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract claims (i) public datasets underrepresent long-context multi-turn traffic (quantified in §3), (ii) we evaluate routing policies on a production trace (Tables 2, 3, 4), and (iii) GORGO variants together sweep every TTFT percentile in every GLM-5.1 window. We are explicit in the introduction and §8 that “GORGO” refers to a family of variants and that the result is regime-dependent (the WildChat replay reverses the ranking).

2. Limitations

Question: Does the paper discuss the limitations of the work?

Answer: [Yes]

Justification: Section 8 groups limitations into statistical scope (single trace, single run per window with derived p99 standard error), operational scope (single-tenant fleet, centralized-proxy SPOF), methodological caveats (gorgo-autotune identifiability failure, ES convergence time), and system scope (homogeneous hardware, no PD-disaggregation, no scheduler-internal signals).

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete proof?

Answer: [N/A]

Justification: The paper presents an additive cost model and an online optimizer with no formal theorems.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results?

Answer: [Yes]

Justification: Section 6 specifies model, regions, hardware, fleet topology, concurrency, trace windows, and per-window GORGO seeding. Section 4 gives Equation 1, the (1+1)-ES configuration ($\sigma_0 = 0.5$, 1/5-rule, hop 16, window 64), and the median-of-rates fit. The reproducibility statement enumerates four canonical paths in the released artifact.

5. Open access to data and code

Question: Does the paper provide open access to the data and code?

Answer: [No]

Justification: The GLM-5.1 production trace is not redistributable under its terms of service. The proxy and policy code is not yet open-sourced at submission time. The public-dataset characterization (LMSYS, WildChat) is fully reproducible with no proprietary inputs.

6. Experimental setting/details

Question: Does the paper specify all the training and test details?

Answer: [Yes]

Justification: Section 6 specifies model, hardware, regions, concurrency, trace windows, max input tokens, and per-window seeding for adaptive policies. ES configuration is given in Section 4.

7. Experiment statistical significance

Question: Does the paper report error bars or appropriate statistical significance information?

Answer: [No]

Justification: Each data point is a single 30-minute window. Section 8 derives an approximate p99 standard error and shows that the W2 p99 spread between gorgo-hillclimb and random is inside one standard error. The W1 sweep and W2 p50/p95 wins are several standard errors clear of zero.

8. Experiments compute resources

Question: Does the paper provide sufficient information on compute resources?

Answer: [Yes]

Justification: Each policy runs on a dedicated 3-replica fleet of L40S:2 SGLang servers

(tensor-parallel 2, 96 GB VRAM per replica). Eight policies $\times$ two windows = 16 fleet-runs, each ~ 30 min wall clock, on three regions. Total GPU-hours: ≈ 24 .

486 **9. Code of ethics**
 487 Question: Does the research conform with the NeurIPS Code of Ethics?
 488 Answer: [Yes]
 489 Justification: The work concerns routing infrastructure. The production trace is used under
 490 terms that permit research replay; we release only aggregated prefix-trie statistics, not
 491 individual requests. No individuals are identifiable from the released aggregates.

492 **10. Broader impacts**
 493 Question: Does the paper discuss potential societal impacts?
 494 Answer: [Yes]
 495 Justification: Routing improvements reduce the energy and dollar cost of serving LLMs
 496 at fixed quality. Lower-cost serving may accelerate deployment in settings where that is
 497 socially harmful; the contributions are infrastructure-level and do not affect model capability.

498 **11. Safeguards**
 499 Question: Does the paper describe safeguards for responsible release?
 500 Answer: [N/A]
 501 Justification: We do not release datasets or models. The released code is a routing proxy
 502 and policy library with no inherent dual-use beyond LLM-serving infrastructure.

503 **12. Licenses**
 504 Question: Are creators or owners of assets properly credited?
 505 Answer: [Yes]
 506 Justification: SGLang, vLLM, AIBrix, LMSYS-Chat-1M, WildChat-4.8M, and Qwen3 are
 507 cited and used under their published licenses.

508 **13. New assets**
 509 Question: Are new assets introduced in the paper well documented?
 510 Answer: [Yes]
 511 Justification: The proxy, policy library, experiment controller, and trace builder are doc-
 512 umented in the repository’s top-level documentation. Spec and manifest files are stored
 513 alongside the controller.

514 **14. Crowdsourcing and research with human subjects**
 515 Question: Does the paper involve crowdsourcing or human subjects?
 516 Answer: [N/A]
 517 Justification: No human subjects.

518 **15. IRB approvals**
 519 Answer: [N/A]
 520 Justification: No human subjects.

521 **16. Declaration of LLM usage**
 522 Question: Does the paper describe the usage of LLMs if it is an important, original, or
 523 non-standard component of the research methodology?
 524 Answer: [N/A]
 525 Justification: LLM serving infrastructure is the object of study. Authors used LLMs for
 526 routine writing assistance only.

527 **Reproducibility Statement**

528 The experiments are driven by a single experiment controller that launches fleets, configures policies,
529 and replays a trace per policy. The four canonical paths in the released artifact are:

- 530 • `specs/policy_matrix_abstract_night.json` — W1 spec (eight policies, manual
531 GORGO starter weights);
- 532 • `specs/policy_matrix_abstract_night_w2.json` — W2 spec (eight policies,
533 GORGO seeded with W1-learned weights);
- 534 • `experiment_runner/policy_matrix_app.py` — experiment controller;
- 535 • `data_processing/build_mooncake_trace.py` — trace builder.

536 W1 and W2 manifests, result CSVs, and summary plots are produced by running the controller
537 against these specs and post-processing per-policy stats with the released analysis scripts. The GLM-
538 5.1 production trace is not redistributable; the public-dataset characterization (LMSYS, WildChat)
539 reproduces with no proprietary inputs.